Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas
IPN

Materia

Abstracción y uso de datos

Integrantes

Adame Barranco Fernanda Harumi

Gallegos Montiel Isaac

Gonzalez Flores Isaac Yael

Índice

Primer Departamental

Ejercicio 1: Hola mundo en POO 3

Ejercicio 2: Suma con parámetros 3

Ejercicio 3: Calculadora básica 3

Ejercicio 4: Sobrecarga de funciones 4

Ejercicio 5: Herencia 4

Ejercicio 6: Herencia y sobreescritura (polimorfismo) 5

Ejercicio 7: Creación de un nuevo dato (POO) 5

Ejercicio 7 bis: Identificar tamaños 6

Ejercicio 8: Promedio con variables independientes 6

Ejercicio 9: Promedio con arreglo 6

Ejercicio 10: Promedio con puntero al arreglo 7

Ejercicio 11: Matrices 7

Ejercicio 12: Nuevo tipo de dato como arreglo 8

Ejercicio 13: Nuevo tipo de dato con puntero al arreglo 8

Ejercicio 14: Recursividad (factorial, Fibonacci y calculadora) 9

Ejercicio 15: Recursividad con fractales 10

Ejercicio 1: Hola mundo en POO

```
class Fundamentos {
public:
    void holaMundo() { cout << "Hola mundo desde POO!" << endl; }
};
```

Desarrollo: Se deja atrás la programación estructurada, en la que las instrucciones se escriben directamente en main. Aquí la instrucción de impresión se encapsula dentro del método holaMundo(), que pertenece a la clase Fundamentos. En main solo se instancia (crea) un objeto de esa clase para invocar su método.

Error común: Olvidar el punto y coma ; al cerrar la declaración de la clase. El compilador no señala el fallo donde ocurre y genera una cascada de errores confusos en las líneas siguientes.

Solución: Cerrar siempre la estructura de la clase con }; al final.

Ejercicio 2: Suma con parámetros

```
class Aritmetica {
public:
    void sumar(double a, double b) { cout << "Suma: " << (a + b) << endl; }
};
```

Desarrollo: El método sumar recibe dos valores (double a, double b) que se le envían desde fuera en el momento de invocarlo. La clase no interactúa con el usuario: solo procesa los datos que se le proporcionan.

Error común: Declarar los métodos sin modificador de acceso. Dentro de una clase, C++ los trata como private por defecto, así que al llamar operacion.sumar() desde main aparece un error de inaccesibilidad.

Solución: Escribir de forma explícita public: antes de los métodos que se llamarán desde fuera de la clase.

Ejercicio 3: Calculadora básica

```
class CalculadoraBasica {
public:
    void sumar(double a, double b) { cout << "Suma: " << (a + b) << endl; }
    void restar(double a, double b) { cout << "Resta: " << (a - b) << endl; }
    void multiplicar(double a, double b) { cout << "Mult: " << (a * b) << endl; }
    void dividir(double a, double b) {
        if(b != 0) cout << "Div: " << (a / b) << endl;
    }
};
```

Desarrollo: Se construyen cuatro módulos independientes: suma, resta, multiplicación y división. Mediante una estructura de control como switch, el usuario elige qué método activar y le envía los dos operandos como datos de entrada.

Error común: Si el usuario ingresa cero como divisor, la división produce un resultado inválido (infinito) o interrumpe el programa.

Solución: Validar con la condición if(b != 0) justo antes de ejecutar la operación a / b.

Ejercicio 4: Sobrecarga de funciones

```
class CalculadoraSobrecarga {
public:
    void sumar() { double a,b; cin >> a >> b; cout << a+b; }
    void sumar(double a, double b) { cout << a+b; }
    void sumar(double a, double b, double c) { cout << a+b+c; }
};
```

Desarrollo: C++ permite reutilizar el mismo identificador (sumar) varias veces dentro de una clase. El compilador examina cuántos argumentos y de qué tipo se envían en la llamada y selecciona automáticamente la versión correcta.

Error común: Crear dos funciones con los mismos parámetros (cantidad y tipo) pero distinto tipo de retorno, por ejemplo int sumar(double a, double b) y void sumar(double a, double b). Esto genera un conflicto de ambigüedad.

Solución: Garantizar que la firma del método (el número o el tipo de argumentos entre paréntesis) sea siempre única.

Ejercicio 5: Herencia

```
class CalculadoraBase {
public:
    double sumar(double a, double b) { return a + b; }
};

class CalculadoraNueva : public CalculadoraBase {
public:
    double multiplicar(double a, double b) { return a * b; }
};
```

Desarrollo: CalculadoraNueva absorbe las capacidades de CalculadoraBase mediante la sintaxis : public. Así el software se organiza de forma jerárquica: se reutiliza el código existente (la suma) y se le añaden nuevas capacidades (la multiplicación) sin modificar la clase original.

Error común: Si la clase base declara variables como private, la clase hija no puede acceder a ellas para realizar sus propias operaciones.

Solución: Cambiar el modificador de esas variables en la clase padre a protected: siguen protegidas del exterior, pero quedan accesibles para las clases derivadas.

Ejercicio 6: Herencia y sobreescritura (polimorfismo)

```
class CalculadoraOriginal {
public:
    virtual int multiplicar(int a, int b) { return a * b; }
};

class CalculadoraSucesiva : public CalculadoraOriginal {
public:
    int multiplicar(int a, int b) override {
        int res = 0;
        for(int i=0; i<abs(b); i++) res += abs(a);
        return res;
    }
};
```

Desarrollo: La lógica de la clase padre se reemplaza por una nueva en la clase hija. Para obtener la multiplicación mediante operaciones sucesivas, el método multiplicar de la hija repite sumas dentro de un ciclo for.

Error común: Olvidar la palabra virtual en el método de la clase padre. Cuando el objeto se maneja mediante punteros o referencias, el programa ejecuta la versión del padre en lugar de la de la hija.

Solución: Declarar como virtual los métodos del padre que se vayan a redefinir con override.

Ejercicio 7: Creación de un nuevo dato (POO)

```
class Auto {
private:
    double precio; int anio;
public:
    void ingresarDatos() { cin >> precio >> anio; }
};
```

Desarrollo: Se modela una estructura compleja (por ejemplo Auto o Persona) agrupando sus características bajo el modificador private. De esta forma, el programa principal no puede modificar los datos directamente: debe pasar por el «puente» controlado que es el método público ingresarDatos().

Error común: Intentar asignar un valor directamente desde main, como en miAuto.precio = 50000;. Esto impide compilar por violación de acceso.

Solución: Respetar la encapsulación invocando métodos o, si se necesita modificar un solo campo, crear funciones de acceso (getters y setters).

Ejercicio 7 bis: Identificar tamaños

```
cout << "Tamano de Auto: " << sizeof(Auto) << " bytes" << endl;
```

Desarrollo: El operador sizeof indica cuánta memoria (en bytes) debe reservar el sistema para un objeto de la clase.

Error común: Creer que mientras más métodos tenga una clase, mayor será su tamaño en bytes.

Solución: Recordar que sizeof solo considera las variables miembro (los datos) más el relleno de alineación (padding). Los métodos no cuentan, porque se almacenan una sola vez en otro segmento de memoria.

Ejercicio 8: Promedio con variables independientes

```
double suma = n1 + n2 + n3 + n4 + n5;
double maximo = n1;
if(n2 > maximo) maximo = n2;
```

Desarrollo: Los cinco datos se almacenan en variables independientes y se suman de forma manual. Para hallar el máximo (y el mínimo) se toma como punto de partida la primera variable y se compara con las otras cuatro mediante condicionales if consecutivos, actualizando el valor cuando corresponda.

Error común: Inicializar maximo en 0 (o minimo en 9999). Si el usuario ingresa solo valores negativos, el máximo quedará en 0 aunque ese valor no pertenezca al conjunto.

Solución: Inicializar siempre las variables de comparación con el primer valor capturado (por ejemplo, maximo = n1;).

Ejercicio 9: Promedio con arreglo

```
double numeros[5];
for(int i = 0; i < 5; i++) {
    suma += numeros[i];
    if(numeros[i] > maximo) maximo = numeros[i];
}
```

Desarrollo: Los datos se guardan de forma contigua en un arreglo, lo que permite delegar en un ciclo for la acumulación de la suma y las comparaciones contra el máximo en una sola pasada.

Error común: Un error de «fuera de rango» al escribir el ciclo como for(int i = 1; i <= 5; i++). En un arreglo de tamaño 5 el índice 5 no existe: se accede a memoria que no pertenece al arreglo.

Solución: Recorrer siempre desde el índice 0 y terminar un valor antes del tamaño, con la condición i < 5.

Ejercicio 10: Promedio con puntero al arreglo

```
double* ptr = numeros;
for(int i = 0; i < 5; i++) {
    suma += *(ptr + i);
}
```

Desarrollo: La variable ptr guarda la dirección de memoria donde comienza el arreglo. Al sumarle el índice (ptr + i) se avanza exactamente la cantidad de bytes necesaria para llegar al siguiente elemento, y *(ptr + i) lo lee sin usar corchetes.

Error común: Incrementar el propio puntero (ptr++) en cada vuelta sin conservar una copia. Al querer recorrer de nuevo el arreglo (por ejemplo, para calcular el máximo), el puntero ya apunta a otra zona de memoria.

Solución: Navegar con *(ptr + i) o ptr[i], de modo que ptr siga apuntando al inicio del arreglo.

Ejercicio 11: Matrices

Desarrollo: Se piden dos operaciones con matrices: multiplicar la matriz A por una constante y calcular el producto A × B. Se emplearon arreglos bidimensionales de tamaño máximo 10×10, y el programa valida que las dimensiones no excedan ese límite.

Para A × constante basta con recorrer la matriz con dos ciclos (filas y columnas) y multiplicar cada elemento por la constante.

Para A × B debe cumplirse que las columnas de A sean iguales a las filas de B. Por ello el programa solo solicita las columnas de B; sus filas se toman de las columnas de A. Cada elemento del resultado es la suma de los productos de una fila de A por una columna de B, de modo que se requieren tres ciclos anidados: i recorre las filas de A, j las columnas de B y k los elementos que se multiplican y suman. El acumulador r se reinicia en cada celda.

```
double A[10][10], B[10][10], K, r;
int f, c, c2, i, j, k;
for (i = 0; i < f; i++) {
    for (j = 0; j < c; j++) cout << A[i][j] * K << " ";
    cout << "\n";
}
for (i = 0; i < f; i++) {
    for (j = 0; j < c2; j++) {
        for (r = 0, k = 0; k < c; k++) r += A[i][k] * B[k][j];
        cout << r << " ";
    }
    cout << "\n";
}
```

Ejercicio 12: Nuevo tipo de dato como arreglo

Desarrollo: Se pide generar una lista de personas con sus coches. Se crearon dos tipos de dato con struct: Coche (marca, modelo y año) y Persona (nombre, edad y un Coche). Un tipo contiene a otro, porque cada persona tiene su coche.

Con esos tipos se declaró un arreglo de 10 personas. El programa pregunta cuántas se capturarán (entre 1 y 10), llena el arreglo con un ciclo y después lo recorre para imprimir la lista. Para acceder a los datos se encadena el operador punto: lista[i].coche.marca significa «la marca del coche de la persona número i».

```
struct Coche { string marca, modelo; int anio; };
struct Persona { string nombre; int edad; Coche coche; };
Persona lista[10];
for (i = 0; i < n; i++)
    cin >> lista[i].nombre >> lista[i].edad >> lista[i].coche.marca
        >> lista[i].coche.modelo >> lista[i].coche.anio;
for (i = 0; i < n; i++)
    cout << lista[i].nombre << ", " << lista[i].edad << " | "
         << lista[i].coche.marca << " " << lista[i].coche.modelo
         << " (" << lista[i].coche.anio << ")\n";
```

Ejercicio 13: Nuevo tipo de dato con puntero al arreglo

Desarrollo: Se parte del ejercicio anterior, pero ahora la lista se recorre de forma indirecta mediante un puntero. Con Persona *p = lista se aprovecha que el nombre de un arreglo equivale a la dirección de su primer elemento, por lo que p apunta a lista[0].

Para capturar, p++ mueve el puntero a la siguiente persona; como p es un puntero a Persona, se usa la flecha -> en lugar del punto. Para imprimir, p regresa al inicio y el ciclo termina al llegar a lista + n, la posición inmediata posterior al último elemento. Son equivalentes p->nombre y (*p).nombre, así como p + i y &lista[i].

```
struct Coche { string marca, modelo; int anio; };
struct Persona { string nombre; int edad; Coche coche; };
Persona lista[10], *p = lista;
for (i = 0; i < n; i++, p++)
    cin >> p->nombre >> p->edad >> p->coche.marca
        >> p->coche.modelo >> p->coche.anio;
for (p = lista; p < lista + n; p++)
    cout << p->nombre << ", " << p->edad << " | "
         << p->coche.marca << " " << p->coche.modelo
         << " (" << p->coche.anio << ")\n";
```

Ejercicio 14: Recursividad (factorial, Fibonacci y calculadora)

Desarrollo: Una función recursiva se llama a sí misma y necesita un caso base que detenga las llamadas.

Factorial: El caso base es n <= 1, que devuelve 1; en caso contrario devuelve n multiplicado por el factorial de n - 1. Se limitó a 20 porque 21! ya no cabe en un unsigned long long.

Fibonacci: El caso base es n <= 1, que devuelve n; en caso contrario suma los dos términos anteriores mediante dos llamadas recursivas. Es fácil de entender, aunque su costo crece de forma exponencial. Para imprimir la serie se invoca la función con 0, 1, 2, etc.

Calculadora: Cada cálculo sucesivo es una nueva llamada recursiva. La función recibe el resultado acumulado, lee la operación y el número, calcula el nuevo valor y pregunta si se desea continuar; si la respuesta es s, se llama a sí misma con el nuevo valor. La división entre cero se ignora y se conserva el valor anterior.

```
unsigned long long factorial(int n) { return n <= 1 ? 1 : n * factorial(n - 1); }

int fibonacci(int n) { return n <= 1 ? n : fibonacci(n - 1) + fibonacci(n - 2); }

void calculadora(double a) {
    char op, seguir;
    double b;
    cin >> op >> b;
    a = op == '+' ? a + b : op == '-' ? a - b : op == '*' ? a * b : b != 0 ? a / b : a;
    cin >> seguir;
    if (seguir == 's') calculadora(a);
}
```

Ejercicio 15: Recursividad con fractales

Desarrollo: Se dibujan dos fractales en consola usando únicamente recursividad. Cada uno se traza sobre una matriz de caracteres que primero se llena de espacios.

Triángulo de Sierpinski: Un triángulo de altura t se divide en tres triángulos de altura t / 2: uno arriba y dos abajo, desplazados t / 2 columnas a cada lado. El caso base es t == 1, que dibuja un solo asterisco. Con altura 16 se necesita un lienzo de 16 filas y 32 columnas, más una columna para el fin de cadena.

Polvo de Cantor: Una línea se rellena con #; después se repite el proceso en el tercio izquierdo y en el derecho, un nivel más abajo, dejando vacío el tercio central. Se usa un ancho de 81 (3^4) para que la división sea exacta en los 5 niveles. El caso base es llegar al nivel 5.

```
char S[16][33], C[5][82];

void sierpinski(int x, int y, int t) {
    if (t == 1) { S[y][x] = '*'; return; }
    sierpinski(x, y, t / 2);
    sierpinski(x - t / 2, y + t / 2, t / 2);
    sierpinski(x + t / 2, y + t / 2, t / 2);
}

void cantor(int x, int largo, int nivel) {
    if (nivel == 5) return;
    memset(&C[nivel][x], '#', largo);
    cantor(x, largo / 3, nivel + 1);
    cantor(x + 2 * largo / 3, largo / 3, nivel + 1);
}

sierpinski(15, 0, 16);
cantor(0, 81, 0);
```